Solution Architecture

Phase 3: Project Design Phase

1. Architectural Goals

The architecture separates user interaction, request handling, AI generation, budget logic and persistence so that each layer has a clear responsibility. The browser never receives the AI provider key or privileged database credentials.

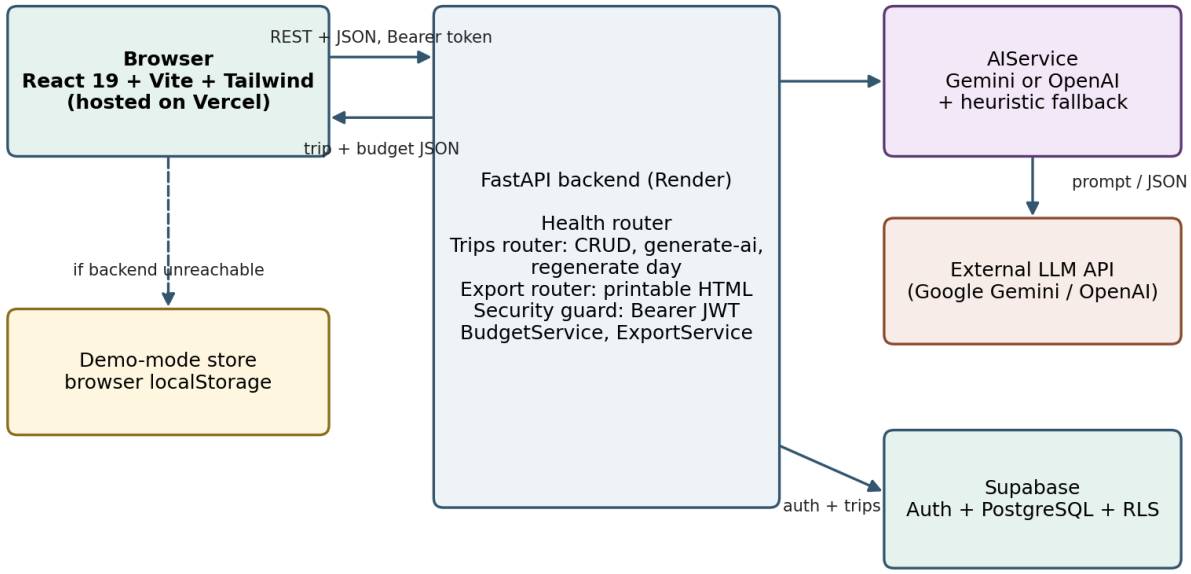


Figure 1. High-level architecture of trvale-book (secret keys stay on the backend).

Figure 1. High-level architecture of trvale-book.

2. Frontend Layer

The React application is organized into pages (landing, login, sign-up, dashboard, wizard, itinerary, saved trips, edit, profile, settings and not-found), reusable components (navbar, sidebar, activity card, day timeline, budget summary, toast, confirmation modal, skeleton loader and demo banner), two context providers (authentication and trip state), a services folder (API client, Supabase client and mock data) and utilities (formatters and print helper). Protected routes redirect signed-out users to the login page unless demo mode is active.

3. API Layer

FastAPI provides GET /health, GET /api/trips, POST /api/trips, GET, PATCH and DELETE /api/trips/{trip_id}, POST /api/trips/generate-ai, POST /api/trips/{trip_id}/days/{day_number}/regenerate and POST /api/trips/{trip_id}/export. A dependency validates the bearer token on every trips route. Errors are returned as HTTP responses with a detail message.

4. AI Service Layer

The AI service builds the prompt, calls the selected provider through httpx with a configurable timeout, parses the JSON reply and hydrates it into the internal trip format. It also contains the heuristic fallback generator and the day-regeneration routine. Keeping this code out of the route handlers allows the model, prompt and validation to change independently.

5. Budget and Export Services

The budget service computes the breakdown and warnings from a trip dictionary. The export service builds a standalone printable HTML page for a trip. Both are pure functions of the trip data, which makes them easy to unit test.

6. Persistence Layer

Supabase stores profiles, trips, trip preferences, itinerary days and itinerary activities with row-level security. The frontend can also keep a demo copy of trips in localStorage when the backend is not reachable.

7. Operational and Security Notes

- Keep the AI key, Supabase key and JWT secret in backend environment settings.
- Set CORS_ORIGINS to the deployed frontend origin, for example https://travel-book-livid.vercel.app.
- Set ENABLE_DEMO_MODE to false when real accounts are used in production.
- Add request timeouts and controlled retries; AI_MAX_RETRIES is declared in the settings but is not yet applied in the provider calls.
- Avoid returning raw exception text to clients in production error responses.
- Monitor model quotas and hosting cold starts.

8. Architecture Limitations

The architecture is appropriate for a prototype. Before a large public launch it needs full database persistence of generated days and activities for every route, rate limiting, observability, and a model-backed day regeneration. These are described as future work rather than current features.